# Design Documentation

# Interactive House

# Revision History

| Date | Version | Description | Author |
|---|---|---|---|
| 01/03/2026 | 1.0 | Initial design item descriptions focusing on requirements document | Katerina Arvay, Dario Ostojic, Andre Sandblom |
| 17/03/2026 | 1.1 | Added clarification of the current hybrid architecture and outlined planned future changes to the system design | Katerina Arvay, Dario Ostojic, Andre Sandblom |
| 23/4/26 | 1.2 | Revised design architecture with backup database. | Katerina Arvay, Dario Ostojic, Andre Sandblom |
| | | | |

# Design item List

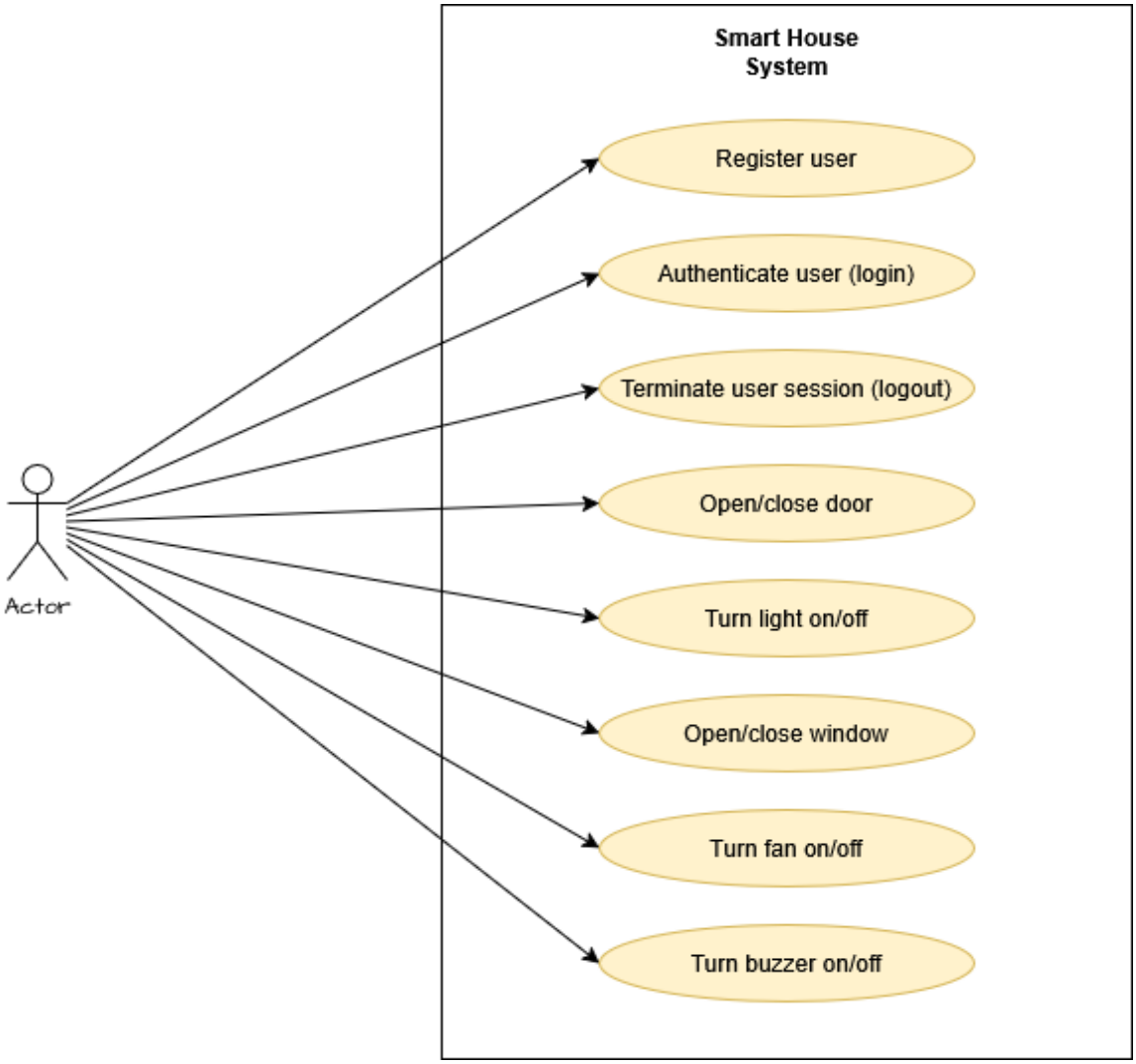
| Requirement Name | Priority |
|---|---|
| D1. Use case diagram | Essential |
| D2. Entity and class diagram | Essential |
| D3. Interaction sequence diagram | Essential |
| D4. State diagram | Essential |

# Design Item Descriptions

## D1. Use case diagram

The user can perform several key actions: registering a new account, logging in to authenticate themselves and logging out to terminate their session.

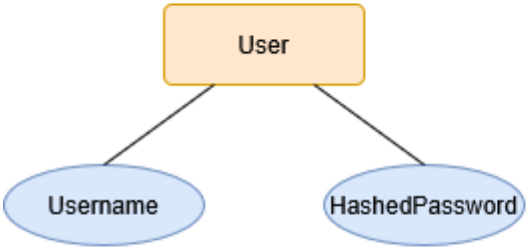
Once authenticated, the user can control various devices in the house. These include opening or closing the door and window, turning lights on or off, activating or deactivating the fan, and controlling the buzzer. Each of these actions corresponds to a specific use case from the system.

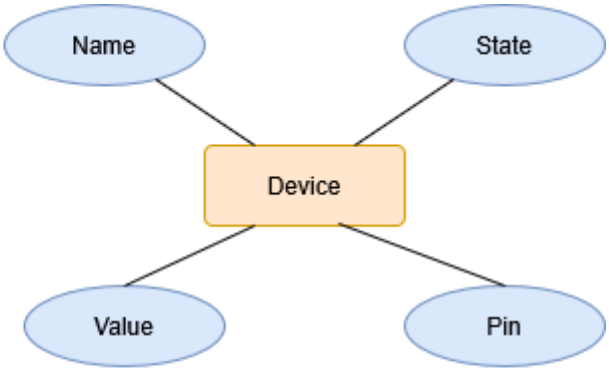


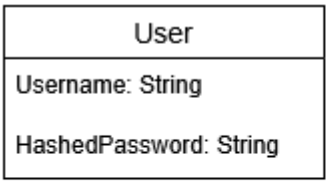

## D2. Entity and class diagram

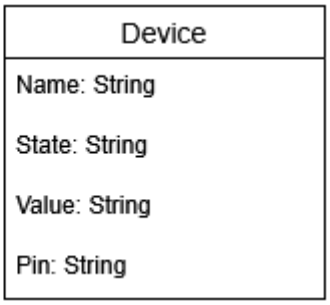
The User entity includes two attributes: Username, which uniquely identifies the user, and HashedPassword, which stores the user's encrypted password for secure authentication.

The Device entity represents smart home devices and contains four attributes: Name to identify the device, State to indicate its status (such as on/off or open/closed), Value to hold any additional relevant data and Pin which corresponds to the hardware control pin.
The system stores and manages entities in a structured backend data model that supports both operational data handling and persistent storage across multiple data storage solutions.




## D3. Interaction sequence diagram

The sequence diagram illustrates how users interact with the Smart House system through web or mobile applications via the backend server.

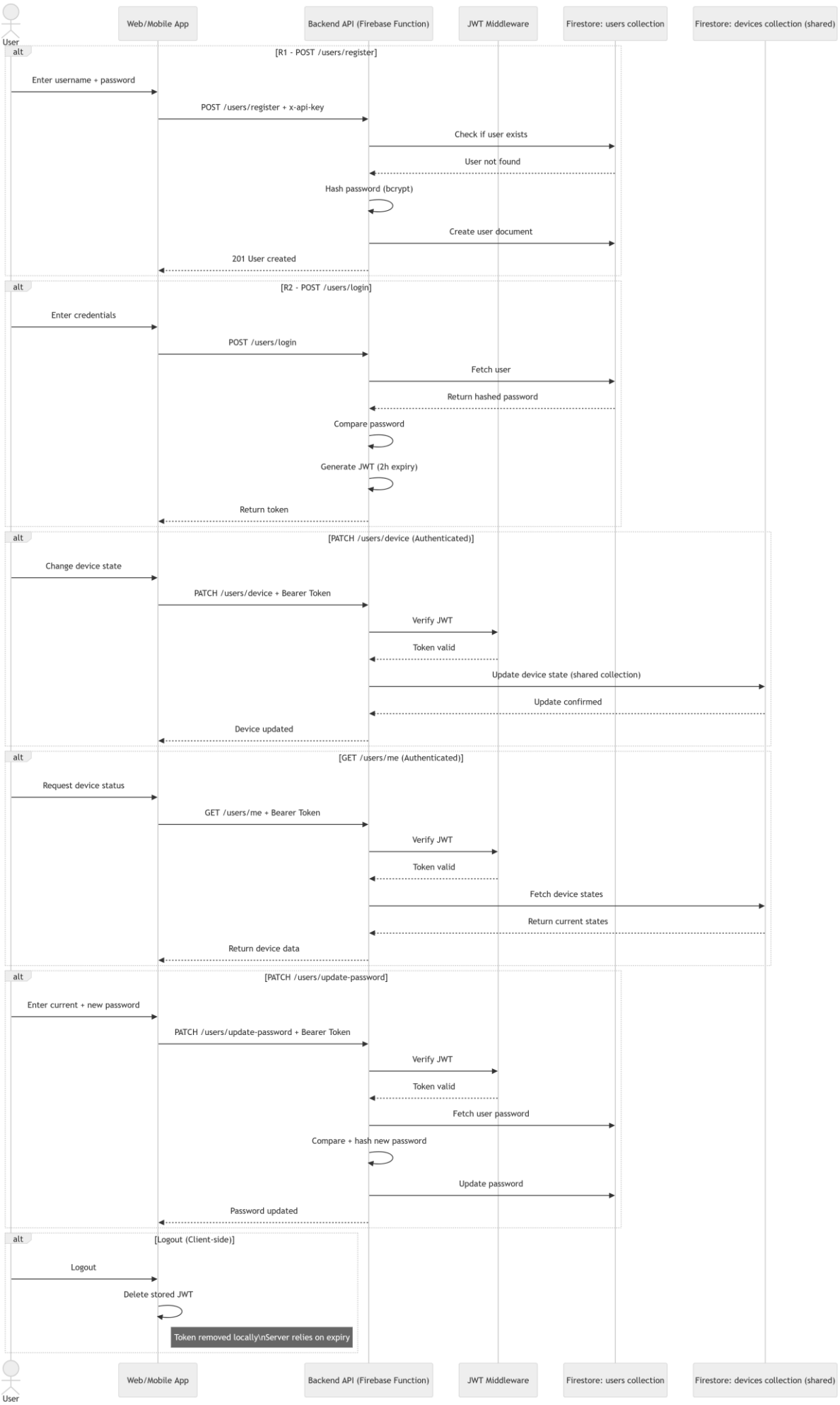
When a new user registers, the client application sends user credentials to the backend API. The server validates the input, securely processes the password, and stores the user information in the system's data storage.

When a user logs in, the client sends authentication credentials to the backend. The server verifies the credentials and, upon success, issues an authentication token that must be included in subsequent requests.

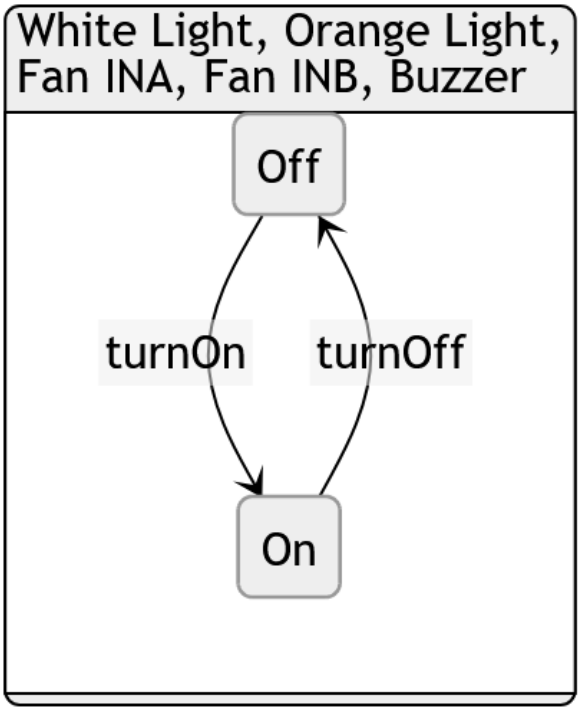
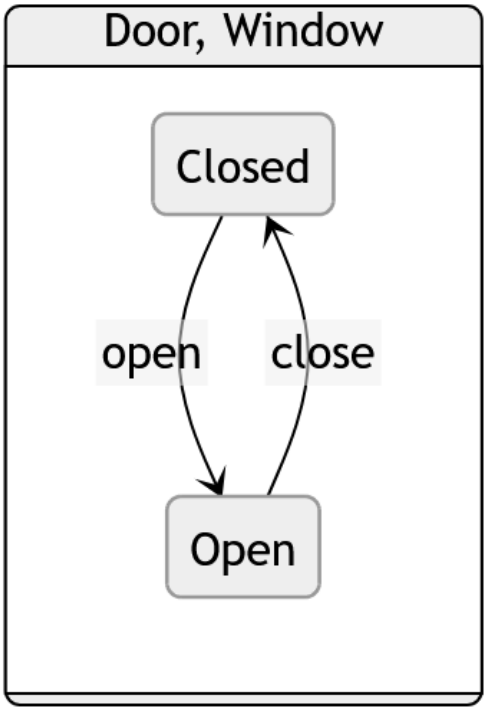
For device control actions (such as turning lights on/off, opening/closing doors or windows, or activating devices), the client sends a request to the backend API along with the authentication token. The backend validates the request, updates the system state, and stores the updated device state in the primary data storage. The system also ensures that critical data is persisted for reliability and recovery purposes.

The backend communicates with the gateway component, which is responsible for forwarding commands to physical devices and reporting back the actual device state. The server updates the stored system state based on confirmed execution results from the gateway.

When retrieving device status, the backend fetches the latest stored state and returns it to the client.
Logout is handled by invalidating or discarding the authentication token, after which the user must log in again to access protected functionality.


User
Web/Mobile App
Backend API (Firebase Function)
JWT Middleware
Firestore: users collection
Firestore: devices collection (shared)
alt
[R1 - POST /users/register]
Enter username + password
POST /users/register + x-api-key
Check if user exists
User not found
Hash password (bcrypt)
Create user document
201 User created
alt
[R2 - POST /users/login]
Enter credentials
POST /users/login
Fetch user
Return hashed password
Compare password
Generate JWT (2h expiry)
Return token
alt
[PATCH /users/device (Authenticated)]
Change device state
PATCH /users/device + Bearer Token
Verify JWT
Token valid
Update device state (shared collection)
Update confirmed
Device updated
alt
[GET /users/me (Authenticated)]
Request device status
GET /users/me + Bearer Token
Verify JWT
Token valid
Fetch device states
Return current states
Return device data
alt
[PATCH /users/update-password]
Enter current + new password
PATCH /users/update-password + Bearer Token
Verify JWT
Token valid
Fetch user password
Compare + hash new password
Update password
Password updated
alt
[Logout (Client-side)]
Logout
Delete stored JWT
Token removed locally\nServer relies on expiry
User
Web/Mobile App
Backend API (Firebase Function)
JWT Middleware
Firestore: users collection
Firestore: devices collection (shared)

This differs slightly from the original design where a centralized backend API handled all interactions, and may be refined in future iterations.

### D4. State diagram

Devices such as lights, fans and the buzzer follow the same On/Off behavior, while the door and window follow an Open/Closed behavior.